

VARIABLES & FACTS

Defining, Prioritizing, and Gathering
Data in Ansible

Professional Reference Documentation

Version 1.0

Table of Contents

1. Introduction to Variables in Ansible
2. Variable Naming and Syntax
 - 2.1 Rules and Conventions
 - 2.2 Referencing Variables with Jinja2
3. Defining Variables at Different Levels
 - 3.1 Play-Level Variables
 - 3.2 Host-Level Variables
 - 3.3 Group-Level Variables
 - 3.4 Role and Task Variables
 - 3.5 Extra Variables from the Command Line
4. Variable Precedence
 - 4.1 The Precedence Hierarchy
 - 4.2 Practical Examples of Precedence
 - 4.3 Common Pitfalls
5. Ansible Facts
 - 5.1 What Facts Are
 - 5.2 Gathering Facts
 - 5.3 Using Facts in Playbooks
 - 5.4 Disabling and Caching Facts
6. Custom Facts
7. Practical Examples
8. Best Practices
9. Quick Reference

1. Introduction to Variables in Ansible

Variables are the mechanism by which Ansible playbooks adapt to different hosts, environments, and use cases. Instead of hard-coding values such as package names, ports, or paths, you define variables and reference them throughout your playbooks. This makes playbooks reusable, maintainable, and safe to run across heterogeneous infrastructure.

Alongside variables, Ansible provides a built-in mechanism for discovering information about managed nodes: **facts**. Facts are structured data such as the operating system, IP addresses, CPU count, memory, mounted filesystems, and available Python interpreter. They are gathered automatically at the start of most plays and are available to every task.

Why this matters:

- **Portability** — the same playbook adapts to Debian, Red Hat, and SUSE systems without branching logic.
- **Reuse** — variables move environment-specific values out of task definitions.
- **Safety** — facts tell you exactly what you are operating on before you change it.
- **Clarity** — a well-named variable expresses intent more clearly than a literal value.

2. Variable Naming and Syntax

2.1 Rules and Conventions

- **Valid names** — letters, digits, and underscores; must begin with a letter or underscore.
- **Case sensitive** — `app_port` and `APP_PORT` are different variables.
- **Convention** — use `lower_snake_case` for most variables. Use `SCREAMING_SNAKE_CASE` only for constants that should never change.
- **Avoid reserved words** — names such as `hosts`, `tasks`, `name`, `vars`, and `roles` have special meaning in playbooks.
- **Avoid Python keywords** — names such as `class`, `for`, `print`, and `type` will cause subtle errors.
- **Prefix with the role or application name** — `nginx_port` is safer than `port` because it cannot collide with another role's variable.

2.2 Referencing Variables with Jinja2

Variables are referenced using Jinja2 expression syntax: `{{ variable_name }}`. Jinja2 is the templating engine embedded in Ansible.

```
# Simple reference
- name: Show the application port
  ansible.builtin.debug:
    msg: "The application listens on {{ app_port }}"

# Reference inside a quoted string
- name: Print full URL
  ansible.builtin.debug:
    msg: "http://{{ inventory_hostname }}:{{ app_port }}/"

# Dictionary access -- two forms, both valid
- name: Access a nested key
  ansible.builtin.debug:
    msg: "{{ nginx_config.port }}" and "{{ nginx_config['port'] }}"

# Default filter -- fall back when a variable is undefined
- name: Use a safe default
  ansible.builtin.debug:
    msg: "{{ app_port | default(8080) }}"
```

Important: Always quote values that begin with {{. In YAML, an unquoted value starting with a brace is interpreted as the start of a dictionary and will fail to parse.

3. Defining Variables at Different Levels

Ansible allows variables to be defined in many places. Where a variable is defined determines both its scope and its precedence relative to other definitions of the same name. The three most important levels for day-to-day work are **play**, **host**, and **group**.

3.1 Play-Level Variables

Play-level variables are declared under the `vars` key of a play. They apply to every task in that play across all targeted hosts and are the simplest place to start.

```
---
- name: Configure the web application
  hosts: webservers
  vars:
    app_name: myapp
    app_port: 8080
    app_root: /opt/myapp
    app_user: deploy
  tasks:
    - name: Create the application directory
      ansible.builtin.file:
        path: "{{ app_root }}"
        state: directory
        owner: "{{ app_user }}"
        mode: '0755'

    - name: Show the configuration
      ansible.builtin.debug:
        msg: "{{ app_name }} will listen on port {{ app_port }}"
```

3.2 Host-Level Variables

Host variables apply to a single managed node. They are typically stored in the `host_vars/` directory adjacent to the inventory, with one file per host. The filename must match the inventory hostname exactly.

```
inventory/
|-- hosts.yml
`-- host_vars/
    |-- web01.example.com.yml
    `-- web02.example.com.yml
```

Example contents of `host_vars/web01.example.com.yml`:

```
---
# Host-specific overrides for web01
app_port: 8080
app_workers: 4
ansible_host: 192.0.2.10

# Custom per-host settings
backup_enabled: true
monitoring_agent: node_exporter
```

3.3 Group-Level Variables

Group variables apply to every host in a named inventory group. They are stored in `group_vars/`, again with one file per group. Because inventory groups can be nested, group variables cascade: a variable set on a parent group is inherited by every host in its children.

```
inventory/
|-- hosts.yml
`-- group_vars/
    |-- all.yml                # every host in the inventory
    |-- webservers.yml         # every member of the webservers group
    |-- databases.yml          # every member of the databases group
    `-- production.yml         # every member of the production parent group
```

Example contents of `group_vars/all.yml`:

```
---
# Applies to every host in the inventory
ntp_server: ntp.example.com
dns_servers:
  - 10.0.0.53
  - 10.0.0.54
timezone: UTC
```

Example contents of `group_vars/webservers.yml`:

```
---
# Applies to every host in the webservers group
app_port: 80
nginx_worker_processes: auto
nginx_client_max_body_size: 10m
```

Why `group_vars` is preferred: keeping variables out of the inventory file separates structure from configuration. The inventory describes *which hosts exist and how they relate*; the `group_vars` and `host_vars` files describe *how they should be configured*.

3.4 Role and Task Variables

Roles bundle their own defaults in `roles/<role>/defaults/main.yml`. These are the lowest-precedence variables and are meant to be overridden by the caller. A second file, `roles/<role>/vars/main.yml`, holds higher-precedence values that should not normally be overridden.

Task-level variables can also be set with the `vars` keyword directly on a task, scoped only to that task:

```
- name: Deploy a specific version for this task only
  ansible.builtin.git:
    repo: https://github.com/example/app.git
    dest: "{{ app_root }}"
    version: "{{ deploy_version }}"
  vars:
    deploy_version: v1.2.3
```

3.5 Extra Variables from the Command Line

Variables passed with `-e` or `--extra-vars` override every other source except task-level `set_fact` in some versions. They are intended for runtime overrides, not for storing permanent configuration.

```
# Single key=value
ansible-playbook site.yml -e app_port=9090

# JSON form (supports nested values and lists)
ansible-playbook site.yml -e '{"app_port": 9090, "debug": true}'

# From a file
ansible-playbook site.yml -e @vars/production.yml
```

4. Variable Precedence

When the same variable is defined in more than one place, Ansible resolves the conflict using a strict precedence order. Understanding this order is essential for debugging unexpected values.

4.1 The Precedence Hierarchy

From **lowest** to **highest** precedence (later entries win):

#	Source	Notes
1	Role defaults roles/x/defaults/main.yml	Lowest precedence; intended to be overridden.
2	Inventory group_vars/all	Applies to every host in the inventory.
3	Inventory group_vars/<parent>	Parent group variables cascade to children.
4	Inventory group_vars/<child>	Child group overrides parent.
5	Inventory host_vars/<host>	Host-specific overrides group values.
6	Play vars vars:	Declared directly under a play.
7	Task vars vars: on a task	Scoped to a single task.
8	set_fact / register	Runtime assignment; survives until the end of the play.
9	Role vars roles/x/vars/main.yml	Higher than defaults but still below extra-vars.
10	Extra vars -e / --extra-vars	Highest precedence; always wins.

Table 1 — Variable precedence from lowest to highest.

Memory aid: more specific and more runtime-scoped wins. A host beats a group. A command-line override beats a file. A task beats a play.

4.2 Practical Examples of Precedence

Example 1 — Group vs. host:

```
# group_vars/webservers.yml
app_port: 80

# host_vars/web01.example.com.yml
app_port: 8080

# On web01, app_port resolves to 8080. On web02 (which has no host_vars
# override), app_port resolves to 80.
```

Example 2 — Parent vs. child group:

```
# group_vars/all.yml
timezone: UTC

# group_vars/production.yml
timezone: America/New_York

# A host in the production group resolves timezone to
# America/New_York. A host outside production uses UTC.
```

Example 3 — Play vars vs. host_vars:

```
# host_vars/web01.example.com.yml
app_port: 8080

# In site.yml:
- hosts: webserver
  vars:
    app_port: 9090
  tasks:
    - debug: msg="{{ app_port }}"    # prints 9090 (play vars win)
```

Example 4 — Extra vars beat everything:

```
# host_vars/web01.example.com.yml
app_port: 8080

# site.yml
- hosts: webserver
  vars:
    app_port: 9090
  tasks:
    - debug: msg="{{ app_port }}"

# Command line:
ansible-playbook site.yml -e app_port=7070    # prints 7070
```

4.3 Common Pitfalls

- **Missing variables.** Always provide a `default()` filter when a variable may not be defined.
- **Typos in filenames.** `host_vars/web01.yml` will not match a host named `web01.example.com`.
- **Unquoted Jinja2.** A value beginning with `{{` must be wrapped in quotes.
- **Role defaults vs. role vars.** Values in `vars/main.yml` are difficult for the caller to override — reserve that file for constants.
- **Overusing extra-vars.** Passing everything on the command line makes runs non-reproducible. Prefer versioned variable files.

5. Ansible Facts

5.1 What Facts Are

Facts are structured data that Ansible discovers about each managed node before executing the tasks in a play. They are collected by the `setup` module and stored in the `ansible_facts` dictionary, which is available to every subsequent task.

Facts typically include:

- **Operating system** — distribution, version, kernel, and family.
- **Network** — interfaces, IP addresses, MAC addresses, and default gateway.
- **Hardware** — CPU architecture, core count, memory, and mounted filesystems.
- **Environment** — hostname, FQDN, date and time, timezone, and environment variables.
- **Runtime** — Python version, available services, and installed packages (on some platforms).

5.2 Gathering Facts

Fact gathering is enabled by default. The play keyword `gather_facts` controls this behaviour.

```
# Default -- facts are gathered automatically
- name: Configure the server
  hosts: webservers
  tasks:
    - name: Show the distribution
      ansible.builtin.debug:
        var: ansible_facts.distribution

# Disable gathering for performance (when facts are not needed)
- name: Run a quick command on every host
  hosts: all
  gather_facts: false
  tasks:
    - name: Ping
      ansible.builtin.ping:

# Gather facts explicitly mid-play
- name: Refresh facts
  hosts: all
  gather_facts: false
  tasks:
    - name: Gather facts only when needed
      ansible.builtin.setup:
```

To inspect all available facts on a host, run the `setup` module directly:

```
ansible web01.example.com -m setup
ansible web01.example.com -m setup -a 'filter=ansible_*_mb'
```

5.3 Using Facts in Playbooks

Facts are referenced either through the `ansible_facts` dictionary (modern form) or as top-level `ansible_*` variables (legacy form). The dictionary form is preferred.

```
# Modern form (recommended)
- name: Install the correct web server for this OS
  ansible.builtin.package:
    name: "{{ 'apache2' if ansible_facts.os_family == 'Debian' else 'httpd' }}"
    state: present

# Legacy top-level form (still works but deprecated)
- name: Show total memory
  ansible.builtin.debug:
    msg: "Total memory: {{ ansible_memtotal_mb }} MB"

# Dictionary form of the same data
- name: Show total memory (modern)
  ansible.builtin.debug:
    msg: "Total memory: {{ ansible_facts.memtotal_mb }} MB"
```

Conditional logic based on facts:

```
- name: Apply Debian-specific configuration
  ansible.builtin.copy:
    src: debian-apt.conf
    dest: /etc/apt/apt.conf.d/99custom
    when: ansible_facts.os_family == 'Debian'

- name: Apply Red Hat-specific configuration
  ansible.builtin.copy:
    src: rhel-yum.conf
    dest: /etc/yum.conf
    when: ansible_facts.os_family == 'RedHat'

- name: Only run on virtual machines
  ansible.builtin.debug:
    msg: "This host is virtualized"
    when: ansible_facts.virtualization_type != 'none'
```

5.4 Disabling and Caching Facts

Fact gathering has a cost: it opens a connection and runs a module on every host. For large inventories or frequently-run playbooks, this overhead can be significant. Two strategies reduce it.

Disable gathering when facts are not needed:

```
- name: Deploy a static file
  hosts: all
  gather_facts: false
  tasks:
    - name: Copy the banner file
      ansible.builtin.copy:
        src: banner.txt
        dest: /etc/issue
```

Enable fact caching in ansible.cfg:

```
[defaults]
gathering          = smart
fact_caching       = jsonfile
fact_caching_connection = /tmp/ansible_facts
fact_caching_timeout = 86400    # 24 hours
```

With `gathering = smart`, Ansible gathers facts only if they are not already present in the cache. This produces a large speed-up on repeated runs against the same hosts.

6. Custom Facts

In addition to built-in facts, Ansible reads custom facts from two locations on the managed node:

- `/etc/ansible/facts.d/*.fact` — system-wide custom facts.
- `~/.ansible/facts.d/*.fact` — per-user custom facts.

Fact files may be JSON, INI, or an executable script that prints JSON to standard output. Custom facts appear under `ansible_facts.ansible_local`.

Example — create a local fact from a playbook:

```
- name: Define a custom local fact
  hosts: webservers
  tasks:
    - name: Ensure the facts directory exists
      ansible.builtin.file:
        path: /etc/ansible/facts.d
        state: directory
        mode: '0755'

    - name: Write a custom fact file
      ansible.builtin.copy:
        dest: /etc/ansible/facts.d/app.fact
        mode: '0644'
        content: |
          {
            "version": "1.2.3",
            "tier": "frontend",
            "region": "us-east-1"
          }

    - name: Re-read facts so the new file is picked up
      ansible.builtin.setup:

    - name: Use the custom fact
      ansible.builtin.debug:
        msg: "App version {{ ansible_facts.ansible_local.app.version }}"
```

7. Practical Examples

Example A — OS-aware package installation:

```
---
- name: Install the web server appropriate for this OS
  hosts: webservers
  become: true

  vars:
    web_package:
      Debian: apache2
      RedHat: httpd
      Suse:  apache2

  tasks:
    - name: Install the web server
      ansible.builtin.package:
        name: "{{ web_package[ansible_facts.os_family] }}"
        state: present
        when: ansible_facts.os_family in web_package
```

Example B — Variable layering across environments:

```
# group_vars/all.yml -- shared values
app_name: myapp
app_user: deploy

# group_vars/staging.yml -- staging overrides
app_port: 8080
debug_enabled: true

# group_vars/production.yml -- production overrides
app_port: 443
debug_enabled: false
ssl_enabled: true

# host_vars/web01.example.com.yml -- host-specific override
app_port: 8443

# The play references only the variable name:
- name: Deploy the application
  hosts: production
  tasks:
    - ansible.builtin.template:
        src: app.conf.j2
        dest: "/etc/{{ app_name }}/app.conf"
      vars:
        port: "{{ app_port }}"
        debug: "{{ debug_enabled }}"
```

Example C — Using facts to build configuration:

```
---
- name: Generate a monitoring configuration from facts
  hosts: all
  become: true

  tasks:
    - name: Write host summary
      ansible.builtin.copy:
        dest: /etc/motd
        mode: '0644'
        content: |
          Hostname:    {{ ansible_facts.hostname }}
          FQDN:        {{ ansible_facts.fqdn }}
          OS:          {{ ansible_facts.distribution }} {{ ansible_facts.distribution_version }}
          Kernel:      {{ ansible_facts.kernel }}
          Arch:         {{ ansible_facts.architecture }}
          CPUs:        {{ ansible_facts.processor_vcpus }}
          Memory:      {{ ansible_facts.memtotal_mb }} MB
          Primary IP:  {{ ansible_facts.default_ipv4.address }}
```

Example D — Fact-driven service management:

```
- name: Ensure the firewall is running, if one is present
  hosts: all
  become: true
  tasks:
    - name: Ensure firewalld is running
      ansible.builtin.service:
        name: firewalld
        state: started
        enabled: true
        when: "'firewalld' in ansible_facts.services"

    - name: Ensure ufw is running
      ansible.builtin.service:
        name: ufw
        state: started
        enabled: true
        when: "'ufw' in ansible_facts.services"
```

8. Best Practices

- **Use group_vars and host_vars, not inline vars.** Keep playbooks focused on tasks and move configuration into the correct variable files.
- **Prefix variable names.** Use `nginx_port` instead of `port` to prevent collisions between roles.
- **Always provide defaults.** Use the `default()` filter or role defaults for any variable that may be missing.
- **Quote Jinja2 expressions.** Write `"{{ var }}"`, not `{{ var }}`, in YAML.
- **Prefer ansible_facts dictionary form.** Use `ansible_facts.os_family` rather than the deprecated `ansible_os_family`.
- **Do not rely on extra-vars for configuration.** Reserve `-e` for runtime toggles; keep permanent values in versioned files.

- **Cache facts in large environments.** Enable a JSON or Redis fact cache to avoid gathering facts on every run.
- **Disable gathering when unnecessary.** Set `gather_facts: false` for plays that only copy static files or run simple commands.
- **Never store secrets in plain variables.** Use `ansible-vault` for passwords, keys, and tokens.
- **Document your variables.** Add comments in `group_vars` and `host_vars` explaining what each variable does and its expected format.

9. Quick Reference

Goal	Syntax
Declare a play variable	<code>vars:\n app_port: 8080</code>
Declare a task variable	<code>- debug:\n vars:\n x: 1</code>
Reference a variable	<code>"{{ app_port }}"</code>
Provide a default	<code>"{{ app_port default(80) }}"</code>
Dictionary access	<code>"{{ config.port }}"</code> or <code>"{{ config['port'] }}"</code>
List index	<code>"{{ servers[0] }}"</code>
String concatenation	<code>"{{ user ~ '@' ~ host }}"</code>
Use a fact	<code>"{{ ansible_facts.os_family }}"</code>
Conditional on a fact	<code>when: ansible_facts.os_family == 'Debian'</code>
Register a task result	<code>- command: date\n register: result</code>
Use a registered result	<code>"{{ result.stdout }}"</code>
Set a runtime variable	<code>set_fact:\n my_var: value</code>
Inspect facts on a host	<code>ansible host -m setup</code>
Filter facts	<code>ansible host -m setup -a 'filter=*mb'</code>
Override on the command line	<code>ansible-playbook site.yml -e app_port=9090</code>

Table 2 — Variables and facts quick reference.

This document describes Ansible Core behaviour as of version 2.14 and later. Variable precedence and fact structure may change between releases; always consult the official documentation for the version in use.